

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-a1a9ba38869e6ea64.jsonl`
- SHA-256: `fdeed332285f93d5ee50d61419e64ed327b3d440e8ef1e924382637b13628bce`
- Source modified: `2026-07-04T00:46:25+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `subagents`
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`
```

Transcript

1. user (2026-07-04T00:44:33.914Z)

Code-review finder. Target: PR #458 of badminton-highlight-indexer. Diff: `cd C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer && gh pr diff 458`. PR branch = `origin/codex/a7-a9-quality-chain`. Files at scratchpad/pr458_fusion_golden.py, pr458_fusion_audio.py.

ANGLE ? reuse. The PR adds new helpers to fusion_golden.py: `is\_full\_fusion\_feature\_file`, `load\_source\_video\_paths`, `attach\_source\_video\_paths`, `\_filter\_specs`. Check whether the codebase already has equivalents:

1. Source-manifest loading: does `backend/eval/golden\_manifest.py` (added in PR #452) already have manifest-loading / path-resolution / source-manifest logic that `load\_source\_video\_paths` / `attach\_source\_video\_paths` duplicates? Grep golden_manifest.py for source/video_path/load. The A2 work (golden_manifest) already rebases paths ? is attach_source_video_paths a competing second path-resolution mechanism?
2. Full-fusion schema detection: does `backend/eval/ablation.py`'s `load\_videos` (which the diff mentions "skips windowing-only schema") ALREADY have a schema-check function that `is\_full\_fusion\_feature\_file` duplicates? Grep ablation.py for schema/candidates/shuttle/person/windowing. If ablation already classifies full-vs-windowing, the two should share one predicate.
3. `\_filter\_specs` (--only): does any sibling eval CLI already have an --only/name-filter helper to reuse?
4. The collector `golden\_source\_videos.json` schema ? is it read anywhere else in the repo already (a canonical loader)?

Only flag concrete duplications naming both locations. Return up to 6 candidates as JSON [{file, line, summary, failure_scenario}] (failure_scenario = maintenance cost). [] if none. Final message = only the JSON array.

2. assistant (2026-07-04T00:44:35.659Z)

I'll analyze the PR #458 diff to identify potential code reuse opportunities. Let me start by getting the diff and examining the files mentioned.

3. user (2026-07-04T00:44:36.683Z)

Exit code 1
unknown flag: --no-index

Usage: gh pr diff [<number> | <url> | <branch>] [flags]

Flags:

--color string	Use color in diff output: {always never auto} (default "auto")
-e, --exclude patterns	Exclude files matching glob patterns from the diff
--name-only	Display only names of changed files
--patch	Display diff in patch format
-w, --web	Open the pull request diff in the browser

4. assistant (2026-07-04T00:44:38.683Z)

Let me get the diff using the correct syntax:

5. user (2026-07-04T00:44:40.898Z)

<persisted-output>

Output too large (51.9KB). Full output saved to: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\tool-results\bp52b7o7p.txt

Preview (first 2KB):

```
diff --git a/SESSION_HANDOFF.md b/SESSION_HANDOFF.md
index 31ea4c14..a50cc8a1 100644
--- a/SESSION_HANDOFF.md
+++ b/SESSION_HANDOFF.md
@@ -2,49 +2,51 @@
```

You are here

-A6 is implemented and open for review as PR #455:

-`https://github.com/Khelsutra/badminton-highlight-indexer/pull/455`.

+A6 PR #455 is merged. This checkout is on `codex/a7-a9-quality-chain`, fast-forwarded to
+`origin/master` at `c5fc27c`, with tracked edits in:

-The branch is `codex/a6-ablation-cli`, rebased onto `origin/master` after A5

-merged as `c8eec7c` / PR #454. It fixes the `python -m backend.eval.ablation`

-circular import by making `ablation.export\_pdf` a lazy wrapper, adds a

-module-entrypoint regression test, and records the n=15 R2/tolF1 contrast in

-`docs/ALPHA\_LAUNCH\_READINESS.md`.

+`- backend/eval/fusion\_golden.py`

+`- backend/eval/fusion\_audio.py`

+`- tests/test\_fusion\_compare.py`

+`- tests/test\_audio\_features.py`

+`- docs/ALPHA\_LAUNCH\_READINESS.md`

+`- SESSION\_HANDOFF.md`

-Validation completed locally on 2026-07-03:

-`ruff check .`, `python -m mypy backend training`, and

-`python run\_all\_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70`

-all passed. Full suite result: 1609 passed, 6 skipped, 85.65% coverage.

+A7/A8/A9 have been run locally against the 15-video corpus, and draft PR #458 is open:

+`https://github.com/Khelsutra/badminton-highlight-indexer/pull/458`. The generated

+feature/model artifacts live under ignored `output/` and `artifacts/` paths and should not

+be committed.

Next steps / open threads

-1. Review/merge PR #455 for A6.

-2. A7 is the next indexer-owned quality item: regenerate or upconvert

- full-fusion feature JSONs for the newer nine golden videos so fusion/person

- /audio analyses can run on all 15 videos. GPU is approved on this machine.

-3. A8 follows A7: rerun `fusion\_compare`, group ablation, and served contrast;

- explicitly promote or reject fusion/person/Gate-A served-default decisions.

...

</persisted-output>

6. assistant (2026-07-04T00:44:42.653Z)

Now let me read the scratchpad files directly to examine the PR changes:

7. user (2026-07-04T00:44:42.949Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\khelsutra-

guru\badminton-highlight-indexer.

8. assistant (2026-07-04T00:44:44.581Z)

Let me check the location of the scratchpad files:

9. user (2026-07-04T00:44:47.366Z)

```
total 72
drwxr-xr-x 1 avidu 197609    0 Jul  4 06:13 .
drwxr-xr-x 1 avidu 197609    0 Jul  3 22:08 ..
-rw-r--r-- 1 avidu 197609 3247 Jul  4 02:16 DEPLOY_REPORT_draft.md
-rw-r--r-- 1 avidu 197609 1274 Jul  4 01:05 gx141_labels.csv
-rw-r--r-- 1 avidu 197609  927 Jul  4 01:05 l6x57_intervals.csv
-rw-r--r-- 1 avidu 197609 2606 Jul  4 02:30 pr_batching_body.md
-rw-r--r-- 1 avidu 197609 6597 Jul  3 22:18 pr451_review.json
-rw-r--r-- 1 avidu 197609 1706 Jul  3 22:32 pr451_signoff.md
-rw-r--r-- 1 avidu 197609 5050 Jul  3 23:10 pr452_review.json
-rw-r--r-- 1 avidu 197609 8111 Jul  4 06:13 pr458_fusion_audio.py
-rw-r--r-- 1 avidu 197609 17976 Jul  4 06:13 pr458_fusion_golden.py
-rwxr-xr-x 1 avidu 197609  1962 Jul  4 01:34 teardown_a1.sh
```

10. assistant (2026-07-04T00:44:49.276Z)

Good. Now let me read those files:

11. user (2026-07-04T00:44:49.727Z)

```
1      """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5      EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6      writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11      Run (on the GPU box):
12      python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14      Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15      `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16      """
17
18      from __future__ import annotations
19
20      import argparse
21      import json
22      import os
23      import time
24      from typing import Any, Dict, List, Optional, Sequence
25
26      from backend.eval import distill_local
27      from backend.eval.calibrate_wasb import load_golden_gts
28      from backend.eval.golden_manifest import load_resolved_manifest
```

```

29 from backend.eval.training import label_candidates
30 from backend.pipeline.detectors import fusion_features as ff
31 from backend.pipeline.detectors import rally_gate
32 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
33 from backend.utils.run_telemetry import RunTelemetry
34
35 # The 5 fps/resolution-invariant shuttle features distill_local already computes.
36 SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
37 # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
38 PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
39 SOURCE_MANIFEST_ENV = "RALLY_GOLDEN_SOURCE_MANIFEST"
40
41
42 def is_full_fusion_feature_file(path: str) -> bool:
43     """True when an existing golden feature JSON already has shuttle+person cue dicts.
44
45     Corpus-growth runs can leave leaner windowing-only JSONs with the same filename
46     pattern.
47     Those are useful for gts/windowing analysis, but they are not the replayable fusion
48     substrate that ``fusion_compare`` and ``ablation`` need.
49     """
50     try:
51         with open(path, encoding="utf-8") as f:
52             data = json.load(f)
53     except (OSError, json.JSONDecodeError, TypeError):
54         return False
55     candidates = data.get("candidates")
56     if not isinstance(candidates, list):
57         return False
58     if not candidates:
59         return True
60     return all(
61         isinstance(candidate, dict)
62         and "shuttle" in candidate
63         and "person" in candidate
64         for candidate in candidates
65     )
66
67
68 def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
69     """Load ``video_id -> local_path`` from the collector golden-source manifest.
70
71     The collector records source/proxy ownership and full-file MD5s. Only entries marked
72     ``present_local`` with a concrete ``local_path`` can drive local GPU extraction.
73     """
74
75     if not source_manifest_path:
76         return {}
77     with open(source_manifest_path, encoding="utf-8") as f:
78         data = json.load(f)
79     entries = data.get("videos", data) if isinstance(data, dict) else data
80     if not isinstance(entries, list):
81         raise ValueError(f"source manifest must contain a videos list: {source_manifest_path}")
82
83     paths: Dict[str, str] = {}
84     for entry in entries:
85         if not isinstance(entry, dict):
86             continue
87         video_id = entry.get("video_id") or entry.get("name")
88         local_path = entry.get("local_path")
89         if not video_id or not local_path:
90             continue
91         if entry.get("source_status", "present_local") != "present_local":

```

```

92         continue
93         paths[str(video_id)] = str(local_path)
94     return paths
95
96
97 def attach_source_video_paths(
98     specs: Sequence[Dict[str, Any]], source_manifest_path: Optional[str]
99 ) -> int:
100     """Attach explicit ``video_path`` fields to manifest specs from the source manifest.
101
102     Returns the number of specs updated. The collector source manifest is authoritative
103     for local source/proxy video location and overrides derived/stale manifest paths.
104     """
105
106     source_paths = load_source_video_paths(source_manifest_path)
107     updated = 0
108     for spec in specs:
109         name = str(spec.get("name", ""))
110         source_path = source_paths.get(name)
111         if not source_path:
112             continue
113         if spec.get("video_path") == source_path:
114             continue
115         spec["video_path"] = source_path
116         updated += 1
117     return updated
118
119
120 def _filter_specs(
121     specs: Sequence[Dict[str, Any]], only: Sequence[str]
122 ) -> List[Dict[str, Any]]:
123     if not only:
124         return list(specs)
125     wanted = list(dict.fromkeys(only))
126     wanted_set = set(wanted)
127     filtered = [spec for spec in specs if str(spec.get("name")) in wanted_set]
128     found = {str(spec.get("name")) for spec in filtered}
129     missing = [name for name in wanted if name not in found]
130     if missing:
131         raise ValueError(f"--only requested unknown video(s): {' '.join(missing)}")
132     return filtered
133
134
135 def _derive_video_path(spec: Dict[str, Any]) -> str:
136     """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
137     golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
138     convention)."""
139     if spec.get("video_path"):
140         return str(spec["video_path"])
141     labels = str(spec["labels"])
142     if labels.endswith(".rallies.csv"):
143         return labels[: -len(".rallies.csv")] + ".MP4"
144     raise ValueError(
145         f"{spec.get('name')}: no video_path and labels {labels!r} isn't *.rallies.csv"
146     )
147
148
149 def extract_video(
150     spec: Dict[str, Any],
151     detector: Optional[Any] = None,
152     frame_skip: int = 3,
153     iou: float = 0.5,
154     log_every_sec: float = 15.0,
155     telemetry: Optional[Any] = None,
156     use_gpu: bool = True,

```

```

156     detector_model: Optional[str] = None,
157     max_gpu_temp_c: Optional[float] = None,
158 ) -> Dict[str, Any]:
159     """One golden video -> a feature record (shuttle + person features per candidate
window).
160
161     Returns ``{name, fps, frame_width,
candidates:[{start,end,shuttle{}},person{}],label}}, gts}``.
162     `detector` (anything with `detections_over_windows`) is injectable for GPU-free
tests;
163     when None a real `PersonDetector` is built (loads torchvision on first use).
164     `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
snapshots
165     from the detection progress callback so a hard kill leaves a forensic runlog.
166     """
167     fps, fw = float(spec["fps"]), float(spec["frame_width"])
168     traj = str(spec["trajectory"])
169     points = TrackNetRunner.parse_trajectory_csv(traj)
170     intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
171     gts = load_golden_gts(str(spec["labels"]))
172     labels = label_candidates(intervals, gts, iou)
173     # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
174     # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
175     axis, net_px, _span = rally_gate.estimate_play_axis(points)
176     video_path = _derive_video_path(spec)
177     if not os.path.isfile(video_path):
178         raise FileNotFoundError(
179             f"{spec.get('name')}: source video not found for fusion extraction:
{video_path}"
180         )
181
182     if detector is None:
183         from backend.pipeline.detectors.person_detector import PersonDetector
184
185         indexing_cfg: Dict[str, Any] = {"use_gpu": use_gpu}
186         if detector_model:
187             indexing_cfg["detector_model"] = detector_model
188         detector = PersonDetector({"indexing": indexing_cfg})
189
190     # Single-pass person detection over ALL candidate windows (open the video ONCE, read
191     # forward ? no per-window seek). Then compute features per window (pure, no GPU).
192     n = len(intervals)
193     win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
194     print(
195         f"[fusion_golden] {spec['name']}: {n} windows ? person-detecting in one
pass...",
196         flush=True,
197     )
198     t0 = time.monotonic()
199     state = {"last": t0}
200
201     def _progress(done: int, total: int) -> None:
202         rec = None
203         if telemetry is not None:
204             rec = telemetry.snapshot(done, total, phase=spec["name"])
205         if max_gpu_temp_c is not None and rec is not None:
206             gpu = rec.get("gpu") or {}
207             temp = gpu.get("temp_c")
208             if temp is not None and float(temp) >= max_gpu_temp_c:
209                 raise RuntimeError(
210                     f"{spec['name']}: GPU temperature {float(temp):.1f}C reached "
211                     f"--max-gpu-temp-c {max_gpu_temp_c:.1f}; aborting before thermal
shutdown"

```

```

212         )
213     now = time.monotonic()
214     if now - state["last"] >= log_every_sec or done >= total:
215         el = now - t0
216         rate = done / el if el > 0 else 0.0
217         eta = (total - done) / rate if rate > 0 else 0.0
218         pct = 100 * done // total if total else 0
219         print(
220             f"[fusion_golden]    {spec['name']}: detect frame {done}/{total} ({pct}%)
221 "
222             f"| {el:.0f}s elapsed | ETA {eta:.0f}s",
223             flush=True,
224         )
225         state["last"] = now
226
227     per_window = detector.detections_over_windows(
228         video_path, win_frames, frame_skip, progress_cb=progress
229     )
230
231     candidates: List[Dict[str, Any]] = []
232     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
233         det = per_window[i]
234         det_fw = det.get("frame_width") or fw
235         det_fh = det.get("frame_height") or 0.0
236         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
237         point
238         # normalisation fusion_features uses (x/width, y/height).
239         if axis == "x":
240             net_norm = net_px / det_fw if det_fw > 0 else 0.5
241         else:
242             net_norm = net_px / det_fh if det_fh > 0 else 0.5
243         sf, ef = win_frames[i]
244         shuttle_in = [p for p in points if sf <= p.frame <= ef]
245         person = ff.compute_person_features(
246             det["frames"],
247             shuttle_in,
248             det_fw,
249             det_fh,
250             court_polygon=None,
251             net=(axis, net_norm),
252         )
253         candidates.append(
254             {
255                 "start": float(s),
256                 "end": float(e),
257                 "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
258                 "person": {k: float(v) for k, v in person.items()},
259                 "label": int(labels[i]),
260             }
261         )
262     return {
263         "name": spec["name"],
264         "fps": fps,
265         "frame_width": fw,
266         "candidates": candidates,
267         "gts": [[float(a), float(b)] for a, b in gts],
268     }
269
270 def run(
271     manifest_path: str,
272     out_dir: str,
273     frame_skip: int = 3,
274     iou: float = 0.5,
275     fresh: bool = False,

```

```

275     smallest_first: bool = False,
276     source_manifest: Optional[str] = None,
277     only: Sequence[str] = (),
278     missing_fusion_only: bool = False,
279     use_gpu: bool = True,
280     detector_model: Optional[str] = None,
281     max_gpu_temp_c: Optional[float] = None,
282 ) -> List[str]:
283     specs = load_resolved_manifest(manifest_path)
284     source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
285     if source_manifest:
286         updated = attach_source_video_paths(specs, source_manifest)
287         print(
288             f"[fusion_golden] attached {updated}/{len(specs)} source video path(s) "
289             f"from {source_manifest}",
290             flush=True,
291         )
292     specs = _filter_specs(specs, only)
293     if smallest_first:
294         # Process small videos first for quick feedback (the slow full match lands
295         # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
296         # row/frame).
297         specs = sorted(
298             specs,
299             key=lambda s: (
300                 os.path.getsize(s["trajectory"])
301                 if os.path.isfile(s.get("trajectory", ""))
302                 else 0
303             ),
304         )
305     os.makedirs(out_dir, exist_ok=True)
306     # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
307     # rotating runlog of velocity + machine health for the post-mortem. Constructing it
308     # break extraction, so guard the build and fall back to no telemetry on any error.
309     telem: Optional[RunTelemetry]
310     try:
311         telem = RunTelemetry(
312             os.path.join(out_dir, "run_telemetry.jsonl"), label="fusion_golden"
313         )
314     except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
315         telem = None
316     written: List[str] = []
317     for vi, spec in enumerate(specs, 1):
318         out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
319         existing = os.path.isfile(out)
320         existing_full = existing and is_full_fusion_feature_file(out)
321         # Resumable: skip a video whose full-fusion JSON already exists. A leaner
322         # windowing-only JSON is stale for A7, so re-extract it even without --fresh.
323         if missing_fusion_only and not fresh and existing_full:
324             print(
325                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "
326                 f"features already present -> {out}",
327                 flush=True,
328             )
329             written.append(out)
330             continue
331         if not fresh and existing and existing_full:
332             print(
333                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted

```



```

334         flush=True,
335     )
336     written.append(out)
337     continue
338 if not fresh and existing and not existing_full:
339     print(
340         f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: existing JSON "
341         "is not full-fusion schema; re-extracting...",
342         flush=True,
343     )
344     print(
345         f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
346         flush=True,
347     )
348     t0 = time.monotonic()
349     if telem is not None:
350         telem.event("video_start", spec["name"])
351     try:
352         data = extract_video(
353             spec,
354             frame_skip=frame_skip,
355             iou=iou,
356             telemetry=telem,
357             use_gpu=use_gpu,
358             detector_model=detector_model,
359             max_gpu_temp_c=max_gpu_temp_c,
360         )
361     except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
362         if telem is not None:
363             telem.event("error", spec["name"], err=str(e))
364         raise
365     n_pos = sum(c["label"] for c in data["candidates"])
366     with open(out, "w", encoding="utf-8") as f:
367         json.dump(data, f, indent=2)
368     if telem is not None:
369         telem.event(
370             "video_done",
371             spec["name"],
372             windows=len(data["candidates"]),
373             positives=n_pos,
374         )
375     print(
376         f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
377         f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
378         f"-> {out}",
379         flush=True,
380     )
381     written.append(out)
382     return written
383
384
385 def main() -> None:
386     ap = argparse.ArgumentParser(
387         description="Extract golden-set shuttle+person features (fusion P3 GPU step)."
388     )
389     ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
390     ap.add_argument("--out-dir", default="output")
391     ap.add_argument("--frame-skip", type=int, default=3)
392     ap.add_argument("--iou", type=float, default=0.5)
393     ap.add_argument(
394         "--fresh", action="store_true", help="re-extract even if a JSON already exists"
395     )

```

```

396         ap.add_argument(
397             "--smallest-first",
398             action="store_true",
399             help="process smaller videos first for quick feedback (LOVO is order-
independent)",
400         )
401         ap.add_argument(
402             "--source-manifest",
403             default=None,
404             help=(
405                 "collector golden_source_videos.json; overrides video paths for source/proxy
"
406                 f"videos (or set {SOURCE_MANIFEST_ENV})"
407             ),
408         )
409         ap.add_argument(
410             "--only",
411             action="append",
412             default=[],
413             metavar="VIDEO_ID",
414             help="process one manifest video by name; repeatable",
415         )
416         ap.add_argument(
417             "--missing-fusion-only",
418             action="store_true",
419             help="skip videos whose existing JSON already has full shuttle+person fusion
schema",
420         )
421         ap.add_argument(
422             "--cpu",
423             action="store_true",
424             help="run torchvision person detection on CPU instead of CUDA",
425         )
426         ap.add_argument(
427             "--detector-model",
428             default=None,
429             choices=[
430                 "fasterrcnn_resnet50_fpn",
431                 "fasterrcnn_mobilenet_v3_large_fpn",
432                 "retinanet_resnet50_fpn",
433             ],
434             help="torchvision detector model; default keeps the configured Faster R-CNN
ResNet",
435         )
436         ap.add_argument(
437             "--max-gpu-temp-c",
438             type=float,
439             default=None,
440             help="abort extraction if telemetry sees GPU temperature at or above this
value",
441         )
442         args = ap.parse_args()
443         run(
444             args.manifest,
445             args.out_dir,
446             args.frame_skip,
447             args.iou,
448             fresh=args.fresh,
449             smallest_first=args.smallest_first,
450             source_manifest=args.source_manifest,
451             only=args.only,
452             missing_fusion_only=args.missing_fusion_only,
453             use_gpu=not args.cpu,
454             detector_model=args.detector_model,
455             max_gpu_temp_c=args.max_gpu_temp_c,

```

```

456         )
457
458
459     if __name__ == "__main__":
460         main()
461

```

12. user (2026-07-04T00:44:50.010Z)

```

1     """Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-
only).
2
3     Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment`
runs
4     `detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio
features per
5     window; `compare_increment` then runs the SHIPPED harness between a shuttle+person
variant
6     and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON
TOP
7     of person, not against raw shuttle. Audio enters only as columns the learned scorer
weights
8     (no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.
9
10    Run (no GPU):
11        python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json
--features-dir output --augment --compare
12    """
13
14    from __future__ import annotations
15
16    import argparse
17    import json
18    import logging
19    import os
20    from typing import Any, Dict, List, Optional, Sequence
21
22    from backend.eval import experiment
23    from backend.eval.fusion_compare import delta_stats
24    from backend.eval.fusion_golden import (
25        PERSON_FEATURE_NAMES,
26        SHUTTLE_FEATURE_NAMES,
27        SOURCE_MANIFEST_ENV,
28        _derive_video_path,
29        _filter_specs,
30        attach_source_video_paths,
31    )
32    from backend.eval.golden_manifest import load_resolved_manifest
33    from backend.pipeline.audio.hit_detector import detect_hits_in_video
34    from backend.pipeline.detectors import audio_features as af
35
36    logger = logging.getLogger(__name__)
37
38
39    def augment(
40        manifest_path: str,
41        features_dir: str,
42        k: float = 2.5,
43        source_manifest: Optional[str] = None,
44        only: Sequence[str] = (),
45    ) -> List[str]:
46        """Add the 3 audio features to each existing ``<name>_golden_features.json``
window."""
47        specs = load_resolved_manifest(manifest_path)
48        source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)

```

```

49     if source_manifest:
50         updated = attach_source_video_paths(specs, source_manifest)
51         print(
52             f"[fusion_audio] attached {updated}/{len(specs)} source video path(s) "
53             f"from {source_manifest}",
54             flush=True,
55         )
56     specs = _filter_specs(specs, only)
57     written: List[str] = []
58     for spec in specs:
59         jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
60         if not os.path.isfile(jp):
61             print(
62                 f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden
first; skipping"
63             )
64             continue
65         video = _derive_video_path(spec)
66         if not os.path.isfile(video):
67             raise FileNotFoundError(
68                 f"{spec.get('name')}: source video not found for audio augmentation:
{video}"
69             )
70         print(
71             f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
72             flush=True,
73         )
74         hits = detect_hits_in_video(video, k=k)
75         with open(jp, encoding="utf-8") as f:
76             data = json.load(f)
77         for c in data["candidates"]:
78             c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
79         with open(jp, "w", encoding="utf-8") as f:
80             json.dump(data, f, indent=2)
81         print(
82             f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows",
83             flush=True,
84         )
85         written.append(jp)
86     return written
87
88
89     def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
90         """Load the FULL-FUSION-SCHEMA feature JSONs into VideoCandidates including the
audio
91         columns (requires `augment` to have run ? raises if a fusion JSON lacks audio).
92
93         Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts) are
SKIPPED, not
94         crashed on, so a mixed corpus loads the fusion subset cleanly (see
``_is_fusion_schema``).
95         The skip is checked BEFORE the audio-presence guard, so a genuine fusion file that
simply
96         hasn't been ``--augment``-ed still raises rather than being silently dropped."""
97         return experiment.load_golden_videos(
98             features_dir,
99             shuttle_columns=SHUTTLE_FEATURE_NAMES,
100             person_columns=PERSON_FEATURE_NAMES,
101             audio_columns=af.AUDIO_FEATURE_NAMES,
102             audio_policy="require",
103             skip_logger=logger,
104             skip_message="fusion_audio.load_videos_with_audio: skipping %s ? windowing-only
"
105             "schema (no shuttle/person cue dicts), not a fusion-feature file",

```

```

106         no_audio_error="{path} has no audio features ? run `--augment` first",
107     )
108
109
110     def compare_increment(
111         videos: List[experiment.VideoCandidates], iou: float = 0.5, threshold: float = 0.5
112     ) -> Dict[str, Any]:
113         """Incremental audio lift: shuttle+person vs shuttle+person+audio (honest
114         LOVO)."""
115         person = experiment.Variant(
116             name="shuttle_person",
117             feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES),
118             threshold=threshold,
119         )
120         person_audio = experiment.Variant(
121             name="shuttle_person_audio",
122             feature_names=list(SHUTTLE_FEATURE_NAMES)
123             + list(PERSON_FEATURE_NAMES)
124             + list(af.AUDIO_FEATURE_NAMES),
125             threshold=threshold,
126         )
127         return experiment.compare(videos, person, person_audio, iou=iou, honest=True)
128
129     def format_result(res: Dict[str, Any], iou: float) -> str:
130         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
131         ds = delta_stats(res)
132         st = ds["sign_test"]
133         lines = [
134             f"=== Fusion P4 incremental AUDIO LOVO (n={res['num_videos']}, IoU={iou},
135             honest) ===",
136             f" shuttle+person          : F1={a['f1']:.3f} P={a['precision']:.3f}
137             R={a['recall']:.3f}",
138             f" shuttle+person+audio : F1={b['f1']:.3f} P={b['precision']:.3f}
139             R={b['recall']:.3f}",
140             f" incremental ?F1 = {d['f1']:.3f} ({res['delta_pct']['f1']:.1f}%)",
141             f" paired ?F1 {ds['mean_delta']:.3f} [95% CI {ds['ci_low']:.3f},
142             {ds['ci_high']:.3f}] "
143             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f}) "
144             f"| CI clears 0: {ds['ci_excludes_zero']}",
145         ]
146         for pv in res["per_video_delta"]:
147             lines.append(f" {pv['video']}<26s} ?F1={pv['f1']:.3f}")
148         return "\n".join(lines)
149
150     def main() -> None:
151         ap = argparse.ArgumentParser(
152             description="Fusion P4 ? incremental audio lift on the golden set."
153         )
154         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
155         ap.add_argument("--features-dir", default="output")
156         ap.add_argument("--k", type=float, default=2.5, help="hit-detector sensitivity")
157         ap.add_argument(
158             "--source-manifest",
159             default=None,
160             help=(
161                 "collector golden_source_videos.json; overrides video paths for source/proxy
162                 "
163                 f"videos (or set {SOURCE_MANIFEST_ENV})"
164             ),
165         )
166         ap.add_argument(
167             "--only",
168             action="append",

```

```

165         default=[],
166         metavar="VIDEO_ID",
167         help="augment one manifest video by name; repeatable",
168     )
169     ap.add_argument(
170         "--augment",
171         action="store_true",
172         help="add audio features to the existing JSONs",
173     )
174     ap.add_argument(
175         "--compare", action="store_true", help="report the incremental audio LOVO ?F1"
176     )
177     ap.add_argument("--iou", type=float, default=0.5)
178     ap.add_argument("--record", action="store_true")
179     args = ap.parse_args()
180
181     if args.augment:
182         augment(
183             args.manifest,
184             args.features_dir,
185             args.k,
186             source_manifest=args.source_manifest,
187             only=args.only,
188         )
189     if args.compare:
190         videos = load_videos_with_audio(args.features_dir)
191         if len(videos) < 2:
192             raise SystemExit(f"need >=2 augmented feature files in {args.features_dir}")
193         res = compare_increment(videos, iou=args.iou)
194         print(format_result(res, args.iou))
195         if args.record:
196             row = experiment.record_experiment(res)
197             print(
198                 f"recorded -> eval_baselines/experiment_leaderboard.json
199                 ({row['timestamp']})"
200             )
201
202     if __name__ == "__main__":
203         main()
204

```

13. assistant (2026-07-04T00:44:52.297Z)

Now let me search for the relevant existing files in the codebase to check for duplication:

14. user (2026-07-04T00:44:53.196Z)

```

1     """Portable golden-corpus manifest loader and smoke checker.
2
3     The owner-side golden manifest is intentionally gitignored and has historically
4     carried absolute paths from whichever box ingested a clip. This module keeps that
5     artifact usable after the workspace moves by resolving stale paths against the
6     current repo layout and failing loudly when required corpus files are absent.
7
8     Default local layout supported:
9     - ../Annotation Setup/Trajectories/<trajectory>.csv
10    - ../Annotation Setup/Golden Labelled/<labels>.csv
11    - output/<labels>.csv
12
13    Extra roots can be supplied with ``--root`` or ``RALLY_GOLDEN_SEARCH_ROOTS``.
14    """
15
16    from __future__ import annotations
17

```

```

18     import argparse
19     import json
20     import os
21     import sys
22     from dataclasses import dataclass
23     from pathlib import Path, PureWindowsPath
24     from typing import Any, Iterable, Sequence
25
26     DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
27     SEARCH_ROOTS_ENV = "RALLY_GOLDEN_SEARCH_ROOTS"
28     DEFAULT_REQUIRED_FIELDS = ("trajectory", "labels")
29
30     _FIELD_DIR = {
31         "trajectory": "Trajectories",
32         "labels": "Golden Labelled",
33     }
34
35
36     @dataclass(frozen=True)
37     class ManifestIssue:
38         """One manifest path problem found by the resolver."""
39
40         name: str
41         field: str
42         path: str
43         reason: str
44
45         def message(self) -> str:
46             shown = f" {self.path!r}" if self.path else ""
47             return f"{self.name}: {self.field}{shown} - {self.reason}"
48
49
50     @dataclass(frozen=True)
51     class ManifestRebase:
52         """One path field changed from a stale manifest value to a local file."""
53
54         name: str
55         field: str
56         old_path: str
57         new_path: str
58
59         def message(self) -> str:
60             return f"{self.name}: {self.field} {self.old_path!r} -> {self.new_path!r}"
61
62
63     class ManifestPathError(RuntimeError):
64         """Raised when a strict manifest load finds missing required files."""
65
66         def __init__(self, manifest_path: str | os.PathLike[str], issues:
67             Sequence[ManifestIssue]):
68             self.manifest_path = os.fspath(manifest_path)
69             self.issues = list(issues)
70             detail = "\n".join(f" - {i.message()}" for i in self.issues[:12])
71             more = "" if len(self.issues) <= 12 else f"\n  ... {len(self.issues) - 12} more"
72             super().__init__(
73                 f"golden manifest path check failed for {self.manifest_path}: "
74                 f"{len(self.issues)} issue(s)\n{detail}{more}"
75             )
76
77     def repo_root() -> Path:
78         """Return the repository root for this module."""
79
80         return Path(__file__).resolve().parents[2]
81

```

```

82
83 def load_manifest(manifest_path: str | os.PathLike[str]) -> list[dict[str, Any]]:
84     """Load the flat golden manifest list and validate its outer shape."""
85
86     path = Path(manifest_path)
87     with path.open(encoding="utf-8") as f:
88         data = json.load(f)
89     if not isinstance(data, list):
90         raise ValueError(f"golden manifest must be a JSON list: {path}")
91     out: list[dict[str, Any]] = []
92     for i, row in enumerate(data):
93         if not isinstance(row, dict):
94             raise ValueError(f"golden manifest row {i} is not an object: {path}")
95         out.append(dict(row))
96     return out
97
98
99 def resolve_manifest_paths(
100     manifest_path: str | os.PathLike[str],
101     *,
102     repo_root_path: str | os.PathLike[str] | None = None,
103     extra_roots: Iterable[str | os.PathLike[str]] = (),
104     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
105 ) -> tuple[list[dict[str, Any]], list[ManifestIssue], list[ManifestRebase]]:
106     """Load and rebase required path fields in a golden manifest.
107
108     Returns ``(resolved_specs, issues, rebased_paths)``. The returned specs preserve
109     all manifest metadata but replace any resolved ``trajectory`` / ``labels`` path
110     with the existing local path. Missing or ambiguous required fields are reported
111     in ``issues``.
112     """
113
114     manifest = load_manifest(manifest_path)
115     return resolve_manifest_specs(
116         manifest,
117         manifest_path,
118         repo_root_path=repo_root_path,
119         extra_roots=extra_roots,
120         required_fields=required_fields,
121     )
122
123
124 def load_resolved_manifest(
125     manifest_path: str | os.PathLike[str],
126     *,
127     repo_root_path: str | os.PathLike[str] | None = None,
128     extra_roots: Iterable[str | os.PathLike[str]] = (),
129     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
130     strict: bool = True,
131 ) -> list[dict[str, Any]]:
132     """Load a manifest with stale paths rebased; optionally raise on missing files."""
133
134     specs, issues, _rebases = resolve_manifest_paths(
135         manifest_path,
136         repo_root_path=repo_root_path,
137         extra_roots=extra_roots,
138         required_fields=required_fields,
139     )
140     if strict and issues:
141         raise ManifestPathError(manifest_path, issues)
142     return specs
143
144
145 def resolve_manifest_specs(
146     specs: Sequence[dict[str, Any]],

```



```

147     manifest_path: str | os.PathLike[str],
148     *,
149     repo_root_path: str | os.PathLike[str] | None = None,
150     extra_roots: Iterable[str | os.PathLike[str]] = (),
151     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
152 ) -> tuple[list[dict[str, Any]], list[ManifestIssue], list[ManifestRebase]]:
153     """Resolve an already-loaded manifest list."""
154
155     manifest = Path(manifest_path)
156     root = Path(repo_root_path).resolve() if repo_root_path else repo_root()
157     roots = [Path(p) for p in _env_roots()]
158     roots.extend(Path(p) for p in extra_roots)
159     required = tuple(required_fields)
160
161     resolved: list[dict[str, Any]] = []
162     issues: list[ManifestIssue] = []
163     rebases: list[ManifestRebase] = []
164     for idx, spec in enumerate(specs):
165         row = dict(spec)
166         name = str(row.get("name") or f"row[{idx}]")
167         for field in required:
168             raw = row.get(field)
169             if not raw:
170                 issues.append(ManifestIssue(name, field, "", "required field missing"))
171                 continue
172             matches = resolve_existing_paths(
173                 str(raw),
174                 field,
175                 manifest_path=manifest,
176                 repo_root_path=root,
177                 extra_roots=roots,
178             )
179             if len(matches) > 1:
180                 found = "; ".join(str(p) for p in matches)
181                 issues.append(
182                     ManifestIssue(
183                         name,
184                         field,
185                         str(raw),
186                         f"ambiguous file matches for basename: {found}",
187                     )
188                 )
189             elif not matches:
190                 issues.append(ManifestIssue(name, field, str(raw), "file not found"))
191             else:
192                 hit = matches[0]
193                 row[field] = str(hit)
194                 if not _same_path(str(raw), hit):
195                     rebases.append(ManifestRebase(name, field, str(raw), str(hit)))
196             resolved.append(row)
197     return resolved, issues, rebases
198
199
200 def resolve_existing_paths(
201     raw_path: str,
202     field: str,
203     *,
204     manifest_path: str | os.PathLike[str],
205     repo_root_path: str | os.PathLike[str],
206     extra_roots: Iterable[str | os.PathLike[str]] = (),
207 ) -> list[Path]:
208     """Return all distinct local files that could satisfy one manifest path field."""
209
210     matches: list[Path] = []
211     for candidate in _candidate_paths(

```

```

212         raw_path,
213         field,
214         Path(manifest_path),
215         Path(repo_root_path),
216         [Path(p) for p in extra_roots],
217     ):
218         if candidate.is_file():
219             matches.append(candidate.resolve())
220     return _dedup_paths(matches)
221
222
223 def resolve_existing_path(
224     raw_path: str,
225     field: str,
226     *,
227     manifest_path: str | os.PathLike[str],
228     repo_root_path: str | os.PathLike[str],
229     extra_roots: Iterable[str | os.PathLike[str]] = (),
230 ) -> Path | None:
231     """Resolve one manifest path field if exactly one local file matches."""
232
233     matches = resolve_existing_paths(
234         raw_path,
235         field,
236         manifest_path=manifest_path,
237         repo_root_path=repo_root_path,
238         extra_roots=extra_roots,
239     )
240     return matches[0] if len(matches) == 1 else None
241
242
243 def _env_roots() -> list[Path]:
244     raw = os.environ.get(SEARCH_ROOTS_ENV, "")
245     if not raw:
246         return []
247     return [Path(p) for p in raw.split(os.pathsep) if p.strip()]
248
249
250 def _basename(raw_path: str) -> str:
251     # PureWindowsPath handles stale Windows paths even when tests run on POSIX.
252     win_name = PureWindowsPath(raw_path).name
253     return win_name or Path(raw_path).name
254
255
256 def _candidate_paths(
257     raw_path: str,
258     field: str,
259     manifest_path: Path,
260     root: Path,
261     extra_roots: Sequence[Path],
262 ) -> list[Path]:
263     basename = _basename(raw_path)
264     raw = Path(os.path.expandvars(os.path.expanduser(raw_path)))
265     manifest_dir = manifest_path.resolve().parent
266     candidates: list[Path] = [raw]
267
268     if not raw.is_absolute():
269         candidates.extend([manifest_dir / raw, root / raw])
270
271     for directory in _candidate_dirs(field, manifest_dir, root, extra_roots):
272         candidates.append(directory / basename)
273     return _dedup_paths(candidates)
274
275
276 def _candidate_dirs(

```

```

277     field: str, manifest_dir: Path, root: Path, extra_roots: Sequence[Path]
278 ) -> list[Path]:
279     field_dir = _FIELD_DIR.get(field)
280     dirs: list[Path] = []
281     if field_dir:
282         dirs.extend(
283             [
284                 root.parent / "Annotation Setup" / field_dir,
285                 root.parent / "Annotation Setup" / "Collect" / field_dir,
286             ]
287         )
288     if field == "labels":
289         dirs.append(root / "output")
290         dirs.append(manifest_dir)
291     dirs.extend(root_dirs for r in extra_roots for root_dirs in _expand_root(r, field))
292     return _dedup_paths(dirs)
293
294
295 def _expand_root(root: Path, field: str) -> list[Path]:
296     field_dir = _FIELD_DIR.get(field)
297     dirs = [root]
298     if field_dir:
299         dirs.extend(
300             [
301                 root / field_dir,
302                 root / "Annotation Setup" / field_dir,
303                 root / "Annotation Setup" / "Collect" / field_dir,
304             ]
305         )
306     if field == "labels":
307         dirs.append(root / "output")
308     return dirs
309
310
311 def _dedup_paths(paths: Iterable[Path]) -> list[Path]:
312     out: list[Path] = []
313     seen: set[str] = set()
314     for path in paths:
315         try:
316             key = os.path.normcase(os.path.abspath(os.fspath(path)))
317         except OSError:
318             key = os.fspath(path)
319         if key in seen:
320             continue
321         seen.add(key)
322         out.append(path)
323     return out
324
325
326 def _same_path(raw: str, resolved: Path) -> bool:
327     try:
328         raw_abs = os.path.abspath(os.path.expanduser(os.path.expandvars(raw)))
329         resolved_abs = os.path.abspath(os.fspath(resolved))
330         return os.path.normcase(raw_abs) == os.path.normcase(resolved_abs)
331     except OSError:
332         return raw == str(resolved)
333
334
335 def write_manifest(path: str | os.PathLike[str], specs: Sequence[dict[str, Any]]) ->
None:
336     """Write a resolved manifest JSON atomically enough for local tooling."""
337
338     out = Path(path)
339     out.parent.mkdir(parents=True, exist_ok=True)
340     tmp = out.with_name(out.name + ".tmp")

```

```

341     with tmp.open("w", encoding="utf-8") as f:
342         json.dump(list(specs), f, indent=2)
343         f.write("\n")
344     os.replace(tmp, out)
345
346
347 def build_parser() -> argparse.ArgumentParser:
348     p = argparse.ArgumentParser(
349         description="Check/rebase the local golden corpus manifest paths."
350     )
351     p.add_argument("--manifest", default=DEFAULT_MANIFEST)
352     p.add_argument(
353         "--repo-root",
354         default=None,
355         help="repo root used for default sibling output/Annotation Setup lookups",
356     )
357     p.add_argument(
358         "--root",
359         action="append",
360         default=[],
361         help=(
362             "additional search root; repeatable. Also accepts roots via "
363             f"{{SEARCH_ROOTS_ENV}} separated by os.pathsep."
364         ),
365     )
366     p.add_argument(
367         "--require",
368         action="append",
369         choices=sorted(_FIELD_DIR),
370         default=None,
371         help="path field to require; repeatable. Defaults to trajectory and labels.",
372     )
373     p.add_argument(
374         "--min-count",
375         type=int,
376         default=None,
377         help="fail unless at least this many manifest rows are present",
378     )
379     p.add_argument(
380         "--write",
381         default=None,
382         help="optional path to write the resolved local manifest JSON",
383     )
384     return p
385
386
387 def main(argv: Sequence[str] | None = None) -> int:
388     args = build_parser().parse_args(argv)
389     required = tuple(args.require or DEFAULT_REQUIRED_FIELDS)
390     try:
391         original = load_manifest(args.manifest)
392         resolved, issues, rebases = resolve_manifest_specs(
393             original,
394             args.manifest,
395             repo_root_path=args.repo_root,
396             extra_roots=args.root,
397             required_fields=required,
398         )
399     except (OSError, ValueError, json.JSONDecodeError) as e:
400         print(f"ERROR: {e}", file=sys.stderr)
401         return 2
402
403     if args.min_count is not None and len(resolved) < args.min_count:
404         issues.append(
405             ManifestIssue(

```

```

406         "manifest",
407         "rows",
408         str(len(resolved)),
409         f"expected at least {args.min_count} row(s)",
410     )
411 )
412
413 if issues:
414     print(
415         f"ERROR: golden manifest path check failed ({len(issues)} issue(s)):",
416         file=sys.stderr,
417     )
418     for issue in issues:
419         print(f" - {issue.message()}", file=sys.stderr)
420     return 2
421
422 print(
423     f"OK: {args.manifest} has {len(resolved)} video(s); "
424     f"{len(resolved) * len(required)} required path(s) present; "
425     f"{len(rebases)} path(s) rebased."
426 )
427 if args.write:
428     write_manifest(args.write, resolved)
429     print(f"wrote resolved manifest: {args.write}")
430 return 0
431
432
433 if __name__ == "__main__":
434     raise SystemExit(main())
435

```

15. user (2026-07-04T00:44:53.945Z)

Found 8 files

```

backend\eval\fusion_golden.py
backend\eval\calibrate_local.py
backend\eval\rally_seg_eval.py
backend\eval\distill_local.py
backend\eval\windowing.py
backend\eval\training.py
backend\eval\experiment.py
backend\eval\gemini_refine.py

```

16. assistant (2026-07-04T00:44:55.828Z)

Now let me check the ablation.py file for schema detection:

17. user (2026-07-04T00:44:56.900Z)

```

22- - **Leave-one-GROUP-out (LOGO)** ? remove a whole producer's columns at once (the owner's
23-   "is this old SIGNAL still relevant?" question at cue granularity).
24-
25:A signal can be a **feature column** (person/audio/shuttle columns the learned scorer
weights) or
26-a **gate knob** (Gate-A's ``min_crossings`` threshold ? ablated by zeroing the knob, NOT by
27-dropping a column). The runner handles both uniformly: ``without_s`` subtracts the columns
AND/OR
28-zeroes the gate knob from ``cfg.full``.
--
32- - a signal **earns its keep** only when removal drops F1 with **CI excluding 0 AND the sign
test
33-   agreeing** (the ``per_user_gate`` bar, in reverse);
34- - **structural signals are exempt from auto-demotion** ? a guard whose value is in failure
modes

```

```

35:     the golden corpus may lack (Gate-A vs held-shuttle FPs) is marked ``structural`` and is
NEVER
36-     flagged dead weight on "no measured lift"; its row reports the (possibly ?0) golden ?
honestly
37-     but tags ``structural_protected``;
38- - the runner emits ``earns_keep`` / ``suggestive`` / ``inert`` / ``structural_protected`` ?
it
--
58-logger = logging.getLogger(__name__)
59-
60-# --- The signal inventory (the things the runner ablates), grouped by producer. -----
61:# Source: distill_local.FEATURE_NAMES (shuttle, includes net_crossings), fusion_features
62:# (person), audio_features (audio). The shuttle group's velocity block + duration are the
63:# learned-column shuttle signals; net_crossings is BOTH a column and the Gate-A gate knob.
64-SHUTTLE_COLUMNS: Tuple[str, ...] = (
65-    "duration",
66-    "peak_velocity",
67-    --
68-    "two_sided_motion",
69-    "mean_player_motion",
70-    "in_court_ratio",
71-    "shuttle_player_colocation",
72-)
73-AUDIO_COLUMNS: Tuple[str, ...] = (
74-    "audio_hit_count",
75-    --
76-)
77-
78-class Signal:
79-    """One unit of ablation ? a feature column (or a named group) and/or a gate knob.
80-
81-    - ``name`` ? table/row identity ("person.colocation" | "gate_a" | "audio").
82-    - ``group`` ? producer ("person" | "gate_a" | "audio" | "shuttle"). LOGO ablates a whole
group.
83-    - ``columns`` ? the persisted feature columns this signal contributes (may be empty for
a
84-    pure gate signal like Gate-A whose column is shared with the shuttle group).
85-    - ``gate_knob`` ? e.g. ``"min_crossings"`` for the Gate-A threshold; ablated by zeroing
it on
86-    the full variant rather than dropping a column.
87-    - ``structural`` ? a guard whose value is in failure modes the golden corpus may lack;
NEVER
88-
89-
90-
91-
92-
93-
94-
95-
96-
97-
98-
99-
100-
101-
102-
103-
104-
105-
106-
107-
108-
109-
110-
111-
112-
113-
114-
115-
116-
117-
118-
119-
120-
121-
122-
123-
124-
125-
126-
127-
128-
129-
130-
131-
132-
133-
134-
135-
136-
137-
138-
139-
140-
141-
142-
143-
144-
145-
146-
147-
148-
149-
150-
151-
152-
153-
154-
155-
156-
157-
158-
159-
160-
161-
162-
163-
164-
165-
166-
167-
168-
169-
170-
171-
172-
173-
174-
175-
176-
177-
178-
179-
180-
181-
182-
183-
184-
185-
186-
187-
188-
189-
190-
191-
192-
193-
194-
195-
196-
197-
198-
199-
200-
201-
202-
203-
204-
205-
206-
207-
208-
209-
210-
211-
212-
213-
214-
215-
216-
217-
218-
219-
220-
221-
222-
223-
224-
225-
226-
227-
228-
229-
230-
231-
232-
233-
234-
235-
236-
237-
238-
239-
240-
241-
242-
243-
244-
245-
246-
247-
248-
249-
250-
251-
252-
253-
254-
255-
256-
257-
258-
259-
260-
261-
262-
263-
264-
265-
266-
267-
268-
269-
270-
271-
272-
273-
274-
275-
276-
277-
278-
279-
280-
281-
282-
283-
284-
285-
286-
287-
288-
289-
290-
291-
292-
293-
294-
295-
296-
297-
298-
299-
300-
301-
302-
303-
304-
305-
306-
307-
308-
309-
310-
311-
312-
313-
314-
315-
316-
317-
318-
319-
320-
321-
322-
323-
324-
325-
326-
327-
328-
329-
330-
331-
332-
333-
334-
335-
336-
337-
338-
339-
340-
341-
342-
343-
344-
345-
346-
347-
348-
349-
350-
351-
352-
353-
354-
355-
356-
357-
358-
359-
360-
361-
362-
363-
364-
365-
366-
367-
368-
369-
370-
371-
372-
373-
374-
375-
376-
377-
378-
379-
380-
381-
382-
383-
384-
385-
386-
387-
388-
389-
390-
391-
392-
393-
394-
395-
396-
397-
398-
399-
400-
401-
402-
403-
404-
405-
406-
407-
408-
409-
410-
411-
412-
413-
414-
415-
416-
417-
418-
419-
420-
421-
422-
423-
424-
425-
426-
427-
428-
429-
430-
431-
432-
433-
434-
435-
436-
437-
438-
439-
440-
441-
442-
443-
444-
445-
446-
447-
448-
449-
450-
451-
452-
453-
454-
455-
456-
457-
458-
459-
460-
461-
462-
463-
464-
465-
466-
467-
468-
469-
470-
471-
472-
473-
474-
475-
476-
477-
478-
479-
480-
481-
482-
483-
484-
485-
486-
487-
488-
489-
490-
491-
492-
493-
494-
495-
496-
497-
498-
499-
500-
501-
502-
503-
504-
505-
506-
507-
508-
509-
510-
511-
512-
513-
514-
515-
516-
517-
518-
519-
520-
521-
522-
523-
524-
525-
526-
527-
528-
529-
530-
531-
532-
533-
534-
535-
536-
537-
538-
539-
540-
541-
542-
543-
544-
545-
546-
547-
548-
549-
550-
551-
552-
553-
554-
555-
556-
557-
558-
559-
560-
561-
562-
563-
564-
565-
566-
567-
568-
569-
570-
571-
572-
573-
574-
575-
576-
577-
578-
579-
580-
581-
582-
583-
584-
585-
586-
587-
588-
589-
590-
591-
592-
593-
594-
595-
596-
597-
598-
599-
600-
601-
602-
603-
604-
605-
606-
607-
608-
609-
610-
611-
612-
613-
614-
615-
616-
617-
618-
619-
620-
621-
622-
623-
624-
625-
626-
627-
628-
629-
630-
631-
632-
633-
634-
635-
636-
637-
638-
639-
640-
641-
642-
643-
644-
645-
646-
647-
648-
649-
650-
651-
652-
653-
654-
655-
656-
657-
658-
659-
660-
661-
662-
663-
664-
665-
666-
667-
668-
669-
670-
671-
672-
673-
674-
675-
676-
677-
678-
679-
680-
681-
682-
683-
684-
685-
686-
687-
688-
689-
690-
691-
692-
693-
694-
695-
696-
697-
698-
699-
700-
701-
702-
703-
704-
705-
706-
707-
708-
709-
710-
711-
712-
713-
714-
715-
716-
717-
718-
719-
720-
721-
722-
723-
724-
725-
726-
727-
728-
729-
730-
731-
732-
733-
734-
735-
736-
737-
738-
739-
740-
741-
742-
743-
744-
745-
746-
747-
748-
749-
750-
751-
752-
753-
754-
755-
756-
757-
758-
759-
760-
761-
762-
763-
764-
765-
766-
767-
768-
769-
770-
771-
772-
773-
774-
775-
776-
777-
778-
779-
780-
781-
782-
783-
784-
785-
786-
787-
788-
789-
790-
791-
792-
793-
794-
795-
796-
797-
798-
799-
800-
801-
802-
803-
804-
805-
806-
807-
808-
809-
810-
811-
812-
813-
814-
815-
816-
817-
818-
819-
820-
821-
822-
823-
824-
825-
826-
827-
828-
829-
830-
831-
832-
833-
834-
835-
836-
837-
838-
839-
840-
841-
842-
843-
844-
845-
846-
847-
848-
849-
850-
851-
852-
853-
854-
855-
856-
857-
858-
859-
860-
861-
862-
863-
864-
865-
866-
867-
868-
869-
870-
871-
872-
873-
874-
875-
876-
877-
878-
879-
880-
881-
882-
883-
884-
885-
886-
887-
888-
889-
890-
891-
892-
893-
894-
895-
896-
897-
898-
899-
900-
901-
902-
903-
904-
905-
906-
907-
908-
909-
910-
911-
912-
913-
914-
915-
916-
917-
918-
919-
920-
921-
922-
923-
924-
925-
926-
927-
928-
929-
930-
931-
932-
933-
934-
935-
936-
937-
938-
939-
940-
941-
942-
943-
944-
945-
946-
947-
948-
949-
950-
951-
952-
953-
954-
955-
956-
957-
958-
959-
960-
961-
962-
963-
964-
965-
966-
967-
968-
969-
970-
971-
972-
973-
974-
975-
976-
977-
978-
979-
980-
981-
982-
983-
984-
985-
986-
987-
988-
989-
990-
991-
992-
993-
994-
995-
996-
997-
998-
999-

```

```

428:         Signal("person", "person", PERSON_COLUMNS),
429:         Signal("audio", "audio", AUDIO_COLUMNS),
430:     ]
431-
432-
433-# ----- corpus I/O
434-
435-
436-def load_videos(features_dir: str) -> List[VideoCandidates]:
437-    """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a ``VideoCandidates``
438-    (intervals + shuttle + person + audio columns + golden gts). Audio columns are optional
439-    (older JSONs predate the audio augment) ? a missing block defaults those columns to 0.0
440-    so
441-    the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
442-
443-    Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema
444-    the
445-    R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus loads
446-    the
447-    ablatably subset cleanly (see ``_is_fusion_schema``)."""
448-    return experiment.load_golden_videos(
449-        features_dir,
450-        shuttle_columns=SHUTTLE_COLUMNS,
451-        person_columns=PERSON_COLUMNS,
452-        audio_columns=AUDIO_COLUMNS,
453-        audio_policy="tolerate-missing",
454-        skip_logger=logger,
455-        skip_message="ablation.load_videos: skipping %s ? windowing-only schema "
456-        "(no shuttle/person cue dicts), not an ablatably fusion-feature file",
457-    )
458-
459-
460-def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
461-    """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)
462-    weighted
463-    by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the learned
464-    path
465-    (distinct from the static Gate-A litmus)."""
466-    cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)

```

18. assistant (2026-07-04T00:44:59.360Z)

Now let me look at the experiment.py file to see if there's already a schema checker:

19. user (2026-07-04T00:45:00.167Z)

```

182-     intervals: List[Interval]
183-     features: Dict[str, List[float]] = field(default_factory=dict)
184-     gts: Optional[List[Interval]] = None
185-
186-
187-def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
188-    """True iff every candidate row carries the per-cue feature dicts the golden-feature
189-    loaders
190-    read (``shuttle`` + ``person``; ``audio`` stays optional ? it is added later by
191-    ``fusion_audio.augment`` and older JSONs predate it).
192-
193-    The velocity-windowing / distill corpus-growth path persists a LEANER
194-
195-
196-    return bool(candidates) and all(
197-        "shuttle" in c and "person" in c for c in candidates
198-    )
199-
200-
201-def load_golden_videos(

```

```

208-     features_dir: str,
209-     *,
210-     shuttle_columns: Sequence[str],
211-     person_columns: Sequence[str],
212-     audio_columns: Sequence[str] = (),
213-     --
219-     (``ablation.load_videos`` / ``fusion_compare.load_videos`` /
220-     ``fusion_audio.load_videos_with_audio``).
221- Reads every FULL-FUSION-SCHEMA ``*_golden_features.json`` in ``features_dir`` into a
222- ``VideoCandidates`` (intervals + the caller-supplied feature columns + golden gts).
223- Leaner
224- velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the
225- corpus-growth path persists) are SKIPPED, not crashed on (see ``_is_fusion_schema``), so
226- a
227- MIXED corpus loads the fusion subset cleanly.
228- The column lists are ALWAYS supplied by the caller ? this loader never hardcodes
229- columns. The
230- three callers differ ONLY in:
231- --
232- videos: List[VideoCandidates] = []
233- for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
234-     with open(path, encoding="utf-8") as f:
235-         d = json.load(f)
236-         cand = d["candidates"]
237-         if not _is_fusion_schema(cand):
238-             skip_logger.info(skip_message, os.path.basename(path))
239-             continue
240-         if audio_policy == "require" and "audio" not in cand[0]:
241-             raise SystemExit(no_audio_error.format(path=path))
242-         intervals = [(float(c["start"]), float(c["end"])) for c in cand]

```

20. assistant (2026-07-04T00:45:02.008Z)

Now let me check if there's already a --only filter helper in other eval CLIs:

21. user (2026-07-04T00:45:02.703Z)

```

backend\eval\fusion_golden.py:120: def _filter_specs(
backend\eval\fusion_golden.py:131:     raise ValueError(f"--only requested unknown video(s):
{'', '.join(missing)}")
backend\eval\fusion_golden.py:292:     specs = _filter_specs(specs, only)
backend\eval\fusion_golden.py:410:         "--only",
backend\eval\fusion_audio.py:29:     _filter_specs,
backend\eval\fusion_audio.py:56:     specs = _filter_specs(specs, only)
backend\eval\fusion_audio.py:163:         "--only",

```

22. assistant (2026-07-04T00:45:05.016Z)

Now let me check if the golden_source_videos.json collector schema is already being read anywhere:

23. user (2026-07-04T00:45:06.685Z)

```

backend/eval/fusion_audio.py:         "collector golden_source_videos.json; overrides video
paths for source/proxy "
backend/eval/fusion_golden.py:         "collector golden_source_videos.json; overrides video
paths for source/proxy "
tests/test_audio_features.py:     source_manifest = workspace / "golden_source_videos.json"
tests/test_audio_features.py:         "schema": "sports-data-
collector.golden_source_videos.v2",
tests/test_fusion_compare.py:     source_manifest = tmp_path / "golden_source_videos.json"

```



```
tests/test_fusion_compare.py: "schema": "sports-data-
collector.golden_source_videos.v2",
```

24. assistant (2026-07-04T00:45:08.563Z)

Let me check for any existing source_manifest reading/loading logic:

25. user (2026-07-04T00:45:09.250Z)

```
Found 17 files
tests\test_fusion_compare.py
backend\eval\fusion_golden.py
tests\test_audio_features.py
tests\test_api_endpoints.py
backend\main.py
backend\storage\gdrive_api.py
tests\test_api_cloud_dispatch.py
backend\storage\gcs.py
tests\test_input_source.py
backend\storage\__init__.py
tools\generate_highlights.py
scripts\run_phase3_eval.py
backend\storage\base.py
backend\storage\gdrive.py
backend\storage\local.py
backend\storage\s3.py
backend\eval\batch.py
```

26. assistant (2026-07-04T00:45:11.290Z)

Now let me check if there's already video_path rebasing logic elsewhere:

27. user (2026-07-04T00:45:11.867Z)

```
Found 2 files
backend\eval\fusion_golden.py
backend\eval\fusion_audio.py
```

28. assistant (2026-07-04T00:45:13.812Z)

Now let me look at the actual diff content to see what the PR adds:

29. user (2026-07-04T00:45:15.649Z)

```
diff --git a/SESSION_HANDOFF.md b/SESSION_HANDOFF.md
index 31ea4c14..a50cc8a1 100644
--- a/SESSION_HANDOFF.md
+++ b/SESSION_HANDOFF.md
@@ -2,49 +2,51 @@
```

You are here

```
-A6 is implemented and open for review as PR #455:
-`https://github.com/Khelsutra/badminton-highlight-indexer/pull/455`.
+A6 PR #455 is merged. This checkout is on `codex/a7-a9-quality-chain`, fast-forwarded to
+`origin/master` at `c5fc27c`, with tracked edits in:
```

```
-The branch is `codex/a6-ablation-cli`, rebased onto `origin/master` after A5
-merged as `c8eec7c` / PR #454. It fixes the `python -m backend.eval.ablation`
-circular import by making `ablation.export_pdf` a lazy wrapper, adds a
-module-entrypoint regression test, and records the n=15 R2/tolF1 contrast in
-`docs/ALPHA_LAUNCH_READINESS.md`.
+- `backend/eval/fusion_golden.py`
+- `backend/eval/fusion_audio.py`
```

```
+ - `tests/test_fusion_compare.py`  
+ - `tests/test_audio_features.py`  
+ - `docs/ALPHA_LAUNCH_READINESS.md`  
+ - `SESSION_HANDOFF.md`
```

-Validation completed locally on 2026-07-03:

```
- `ruff check .`, `python -m mypy backend training`, and  
- `python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70`  
- all passed. Full suite result: 1609 passed, 6 skipped, 85.65% coverage.  
+ A7/A8/A9 have been run locally against the 15-video corpus, and draft PR #458 is open:  
+ `https://github.com/Khelsutra/badminton-highlight-indexer/pull/458`. The generated  
+ feature/model artifacts live under ignored `output/` and `artifacts/` paths and should not  
+ be committed.
```

Next steps / open threads

```
-1. Review/merge PR #455 for A6.  
-2. A7 is the next indexer-owned quality item: regenerate or upconvert  
-   full-fusion feature JSONs for the newer nine golden videos so fusion/person  
-   /audio analyses can run on all 15 videos. GPU is approved on this machine.  
-3. A8 follows A7: rerun `fusion_compare`, group ablation, and served contrast;  
-   explicitly promote or reject fusion/person/Gate-A served-default decisions.  
-4. A3 is owned by another agent; do not duplicate that collector/vault work.  
+1. Address review comments on draft PR #458; convert to ready only after owner/reviewer agrees  
+   the A9 TemporalMaxer scope question is settled.  
+2. Decide whether A9 is considered complete with the current Gen-0 non-serving shadow  
+   bundle or whether TemporalMaxer needs a follow-up scorer-swap PR.  
+3. Do not touch A1; another agent owns it.  
+4. A3 is in collector PR #62 and records all 15 source videos; the two Boxhill originals  
+   were verified on `F:\`.
```

Ramp-up kit

```
- `docs/ALPHA_LAUNCH_READINESS.md`  
-- `docs/data_pipeline/GOLDEN_VIDEOS.md`  
-- `docs/R2_EVAL_METRIC_DESIGN.md`  
-- `backend/eval/ablation.py`  
-- `backend/eval/ablation_pdf.py`  
-- `backend/eval/rally_seg_eval.py`  
-- `tests/test_ablation.py`  
-- `output/human_lovo_manifest.json`  
+ `backend/eval/fusion_golden.py`  
+ `backend/eval/fusion_audio.py`  
+ `training/gen0/harness.py`  
+ `eval_baselines/heuristic_lovo_n15.json`  
+ `output/a7_remaining_default.out.log`  
+ `output/a8_audio_augment.out.log`  
+ `output/a6_r2_ablation.json`  
+ `artifacts/rally_tal_gen0/0.1.1-n15-shadow.20260704/manifest.json`
```

Key decisions

```
-- R2/tolF1 is supported for the baseline-off to served milestone contrast on  
- the 15-video corpus: `Delta tolF1 = +0.225`, CI `[+0.165, +0.283]`,  
- sign `14/0`, `p = 0.000`.  
-- This does not approve fusion/person/audio default changes. Current full-fusion  
- ablation still runs on only six videos because the newer nine do not yet have  
- full-fusion feature schema.  
-- Branching discipline remains important: always fetch/pull first, branch from  
- `origin/master`, stage explicit paths, and verify `origin/master..HEAD` before  
- pushing.  
+ A7: full-fusion schema and audio features are present locally for all 15 videos.  
+ A8: person fusion and audio are suggestive but not statistically clean; do not promote  
+ either as a default. Gate A regresses under served windowing; keep it opt-in/analysis-only.  
+ R2/cap/inactivity remains the strongest quality result on n15: F1 `0.237 -> 0.472`,
```

```

+ tolF1 `0.138 -> 0.364`, merge rate `0.638 -> 0.135`; net over-seg guard passes.
+- A9: Gen-0 LogReg/TAL n15 shadow bundle beats the heuristic mean (`0.551 > 0.4456`)
+ but is non-serving (`9/15` paired wins, `p=0.3036`; #172 and WASB C3 still block).
+- Branching discipline remains important: fetch/pull first, stage explicit paths, and
+ verify `origin/master..HEAD` contains only this branch's commits before pushing.
diff --git a/backend/eval/fusion_audio.py b/backend/eval/fusion_audio.py
index b5c65ec4..7dff0c94 100644
--- a/backend/eval/fusion_audio.py
+++ b/backend/eval/fusion_audio.py
@@ -17,14 +17,17 @@
     import json
     import logging
     import os
-from typing import Any, Dict, List
+from typing import Any, Dict, List, Optional, Sequence

     from backend.eval import experiment
     from backend.eval.fusion_compare import delta_stats
     from backend.eval.fusion_golden import (
         PERSON_FEATURE_NAMES,
         SHUTTLE_FEATURE_NAMES,
+     SOURCE_MANIFEST_ENV,
         _derive_video_path,
+     _filter_specs,
+     attach_source_video_paths,
     )
     from backend.eval.golden_manifest import load_resolved_manifest
     from backend.pipeline.audio.hit_detector import detect_hits_in_video
@@ -33,9 +36,24 @@
     logger = logging.getLogger(__name__)

-def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
+def augment(
+    manifest_path: str,
+    features_dir: str,
+    k: float = 2.5,
+    source_manifest: Optional[str] = None,
+    only: Sequence[str] = (),
+) -> List[str]:
     """Add the 3 audio features to each existing ``<name>_golden_features.json`` window."""
     specs = load_resolved_manifest(manifest_path)
+    source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
+    if source_manifest:
+        updated = attach_source_video_paths(specs, source_manifest)
+        print(
+            f"[fusion_audio] attached {updated}/{len(specs)} source video path(s) "
+            f"from {source_manifest}",
+            flush=True,
+        )
+    specs = _filter_specs(specs, only)
     written: List[str] = []
     for spec in specs:
         jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
@@ -45,6 +63,10 @@
@@ def augment(manifest_path: str, features_dir: str, k: float = 2.5) ->
List[str]:
    )
    continue
    video = _derive_video_path(spec)
+    if not os.path.isfile(video):
+        raise FileNotFoundError(
+            f"{spec.get('name')}: source video not found for audio augmentation: {video}"
+        )
    print(
        f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",

```

```
flush=True,
```

30. assistant (2026-07-04T00:45:17.559Z)

Let me view the full diff for the fusion_golden.py additions:

31. user (2026-07-04T00:45:19.933Z)

```
diff --git a/backend/eval/fusion_golden.py b/backend/eval/fusion_golden.py
index aee6d7e5..0666a0ca 100644
--- a/backend/eval/fusion_golden.py
+++ b/backend/eval/fusion_golden.py
@@ -21,7 +21,7 @@
     import json
     import os
     import time
-from typing import Any, Dict, List, Optional
+from typing import Any, Dict, List, Optional, Sequence

     from backend.eval import distill_local
     from backend.eval.calibrate_wasb import load_golden_gts
@@ -36,6 +36,100 @@
     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
+SOURCE_MANIFEST_ENV = "RALLY_GOLDEN_SOURCE_MANIFEST"
+
+
+def is_full_fusion_feature_file(path: str) -> bool:
+    """True when an existing golden feature JSON already has shuttle+person cue dicts.
+
+    Corpus-growth runs can leave leaner windowing-only JSONs with the same filename pattern.
+    Those are useful for gts/windowing analysis, but they are not the replayable fusion
+    substrate that ``fusion_compare`` and ``ablation`` need.
+
+    """
+
+    try:
+        with open(path, encoding="utf-8") as f:
+            data = json.load(f)
+    except (OSError, json.JSONDecodeError, TypeError):
+        return False
+    candidates = data.get("candidates")
+    if not isinstance(candidates, list):
+        return False
+    if not candidates:
+        return True
+    return all(
+        isinstance(candidate, dict)
+        and "shuttle" in candidate
+        and "person" in candidate
+        for candidate in candidates
+    )
+
+def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
+    """Load ``video_id -> local_path`` from the collector golden-source manifest.
+
+    The collector records source/proxy ownership and full-file MD5s. Only entries marked
+    ``present_local`` with a concrete ``local_path`` can drive local GPU extraction.
+
+    """
+
+    if not source_manifest_path:
+        return {}
+    with open(source_manifest_path, encoding="utf-8") as f:
+        data = json.load(f)
```

```

+     entries = data.get("videos", data) if isinstance(data, dict) else data
+     if not isinstance(entries, list):
+         raise ValueError(f"source manifest must contain a videos list: {source_manifest_path}")
+
+     paths: Dict[str, str] = {}
+     for entry in entries:
+         if not isinstance(entry, dict):
+             continue
+         video_id = entry.get("video_id") or entry.get("name")
+         local_path = entry.get("local_path")
+         if not video_id or not local_path:
+             continue
+         if entry.get("source_status", "present_local") != "present_local":
+             continue
+         paths[str(video_id)] = str(local_path)
+     return paths
+
+
+def attach_source_video_paths(
+    specs: Sequence[Dict[str, Any]], source_manifest_path: Optional[str]
+) -> int:
+    """Attach explicit ``video_path`` fields to manifest specs from the source manifest.
+
+    Returns the number of specs updated. The collector source manifest is authoritative
+    for local source/proxy video location and overrides derived/stale manifest paths.
+    """
+
+    source_paths = load_source_video_paths(source_manifest_path)
+    updated = 0
+    for spec in specs:
+        name = str(spec.get("name", ""))
+        source_path = source_paths.get(name)
+        if not source_path:
+            continue
+        if spec.get("video_path") == source_path:
+            continue
+        spec["video_path"] = source_path
+        updated += 1
+    return updated
+
+
+def _filter_specs(
+    specs: Sequence[Dict[str, Any]], only: Sequence[str]
+) -> List[Dict[str, Any]]:
+    if not only:
+        return list(specs)
+    wanted = list(dict.fromkeys(only))
+    wanted_set = set(wanted)
+    filtered = [spec for spec in specs if str(spec.get("name")) in wanted_set]
+    found = {str(spec.get("name")) for spec in filtered}
+    missing = [name for name in wanted if name not in found]
+    if missing:
+        raise ValueError(f"--only requested unknown video(s): {' '.join(missing)}")
+    return filtered
+
+
+def _derive_video_path(spec: Dict[str, Any]) -> str:
@@ -58,6 +152,9 @@ def extract_video(
    iou: float = 0.5,
    log_every_sec: float = 15.0,
    telemetry: Optional[Any] = None,
+    use_gpu: bool = True,
+    detector_model: Optional[str] = None,
+    max_gpu_temp_c: Optional[float] = None,
) -> Dict[str, Any]:

```

```

"""One golden video -> a feature record (shuttle + person features per candidate window).

@@ -77,11 +174,18 @@ def extract_video(
    # (parity with the shuttle net-crossing gate) rather than hard-coding an orientation.
    axis, net_px, _span = rally_gate.estimate_play_axis(points)
    video_path = _derive_video_path(spec)
+   if not os.path.isfile(video_path):
+       raise FileNotFoundError(
+           f"{spec.get('name')}: source video not found for fusion extraction: {video_path}"
+       )

    if detector is None:
        from backend.pipeline.detectors.person_detector import PersonDetector

-       detector = PersonDetector({"indexing": {"use_gpu": True}})
+       indexing_cfg: Dict[str, Any] = {"use_gpu": use_gpu}
+       if detector_model:
+           indexing_cfg["detector_model"] = detector_model
+       detector = PersonDetector({"indexing": indexing_cfg})

    # Single-pass person detection over ALL candidate windows (open the video ONCE, read
    # forward ? no per-window seek). Then compute features per window (pure, no GPU).
@@ -95,8 +199,17 @@ def extract_video(
    state = {"last": t0}

    def _progress(done: int, total: int) -> None:
+       rec = None
        if telemetry is not None:
            telemetry.snapshot(done, total, phase=spec["name"])
            rec = telemetry.snapshot(done, total, phase=spec["name"])
+       if max_gpu_temp_c is not None and rec is not None:
+           gpu = rec.get("gpu") or {}
+           temp = gpu.get("temp_c")
+           if temp is not None and float(temp) >= max_gpu_temp_c:
+               raise RuntimeError(
+                   f"{spec['name']}: GPU temperature {float(temp):.1f}C reached "
+                   f"--max-gpu-temp-c {max_gpu_temp_c:.1f}; aborting before thermal shutdown"
+               )
            now = time.monotonic()
            if now - state["last"] >= log_every_sec or done >= total:
                el = now - t0
@@ -160,8 +273,23 @@ def run(
    iou: float = 0.5,
    fresh: bool = False,
    smallest_first: bool = False,
+   source_manifest: Optional[str] = None,
+   only: Sequence[str] = (),
+   missing_fusion_only: bool = False,
+   use_gpu: bool = True,
+   detector_model: Optional[str] = None,
+   max_gpu_temp_c: Optional[float] = None,
) -> List[str]:
    specs = load_resolved_manifest(manifest_path)
+   source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
+   if source_manifest:
+       updated = attach_source_video_paths(specs, source_manifest)
+       print(
+           f"[fusion_golden] attached {updated}/{len(specs)} source video path(s) "
+           f"from {source_manifest}",
+           flush=True,
+       )
    specs = _filter_specs(specs, only)
    if smallest_first:
        # Process small videos first for quick feedback (the slow full match lands last);
        # the LOVO result is order-independent. Proxy size by the trajectory CSV (one

```

```

row/frame).
@@ -187,9 +315,19 @@ def run(
    written: List[str] = []
    for vi, spec in enumerate(specs, 1):
        out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
        # Resumable: skip a video whose JSON already exists (a re-run picks up where a prior
        # run left off instead of re-detecting). --fresh forces a redo.
        if not fresh and os.path.isfile(out):
+       existing = os.path.isfile(out)
+       existing_full = existing and is_full_fusion_feature_file(out)
+       # Resumable: skip a video whose full-fusion JSON already exists. A leaner
+       # windowing-only JSON is stale for A7, so re-extract it even without --fresh.
+       if missing_fusion_only and not fresh and existing_full:
+           print(
+               f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "

```

32. assistant (2026-07-04T00:45:22.677Z)

Now let me check if there are duplications in existing code by looking at what experiment.py already does:

33. user (2026-07-04T00:45:23.489Z)

```

180
181     name: str
182     intervals: List[Interval]
183     features: Dict[str, List[float]] = field(default_factory=dict)
184     gts: Optional[List[Interval]] = None
185
186
187     def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
188         """True iff every candidate row carries the per-cue feature dicts the golden-feature
loaders
189         read (`shuttle` + `person`; `audio` stays optional ? it is added later by
190         `fusion_audio.augment` and older JSONs predate it).
191
192         The velocity-windowing / distill corpus-growth path persists a LEANER
193         ``*_golden_features.json`` row (`start`/`end` + velocity stats, no cue dicts)
for clips
194         that never went through fusion feature extraction. The loaders that build cue-rich
195         ``VideoCandidates`` from these files ? ``ablation.load_videos``,
``fusion_compare.load_videos``
196         and ``fusion_audio.load_videos_with_audio`` ? SKIP those rather than crash on
``c["shuttle"]``,
197         so a MIXED corpus (some fusion files, some windowing-only) loads the fusion subset
cleanly.
198         An empty candidate list is not a fusion file either.
199
200         Single source of truth for the three loaders (consolidated 2026-06-18 after PRs
#209/#210
201         each landed an independent copy)."""
202         return bool(candidates) and all(
203             "shuttle" in c and "person" in c for c in candidates
204         )
205
206
207     def load_golden_videos(
208         features_dir: str,
209         *,
210         shuttle_columns: Sequence[str],
211         person_columns: Sequence[str],
212         audio_columns: Sequence[str] = (),
213         audio_policy: str = "omit",
214         skip_logger: logging.Logger = logger,
215         skip_message: str,

```

```

216         no_audio_error: str = "{path} has no audio features",
217     ) -> List[VideoCandidates]:
218         """Shared golden-feature corpus loader behind the three structurally-identical
loaders
219         (`ablation.load_videos` / `fusion_compare.load_videos` /
`fusion_audio.load_videos_with_audio`).
220
221         Reads every FULL-FUSION-SCHEMA `*_golden_features.json` in `features_dir` into a
222         `VideoCandidates` (intervals + the caller-supplied feature columns + golden gts).
Leaner
223         velocity-windowing-only files (no `shuttle`/`person` cue dicts ? the schema the
224         corpus-growth path persists) are SKIPPED, not crashed on (see
`_is_fusion_schema`), so a
225         MIXED corpus loads the fusion subset cleanly.
226
227         The column lists are ALWAYS supplied by the caller ? this loader never hardcodes
columns. The
228         three callers differ ONLY in:
229
230         - `audio_policy` ? how the audio block is handled, checked BEFORE reading any
audio column
231         so a windowing-only file is skipped before either branch applies:
232         * `omit` ? no audio columns are loaded (`audio_columns`
ignored);
233         * `tolerate-missing` ? audio columns load, a missing block defaults them
to 0.0;
234         * `require` ? a fusion file lacking audio raises `SystemExit`
(via
235         `no_audio_error` formatted with `path`), mirroring "run --augment
first".
236         - `skip_logger` / `skip_message` ? the per-caller logger + `%s`-templated
info line
237         emitted when a windowing-only file is skipped (kept byte-identical per caller).
238         """
239         if audio_policy not in ("omit", "tolerate-missing", "require"):
240             raise ValueError(
241                 f"audio_policy must be 'omit'/'tolerate-missing'/'require', got
{audio_policy!r}"
242             )
243         videos: List[VideoCandidates] = []
244         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
245             with open(path, encoding="utf-8") as f:
246                 d = json.load(f)
247                 cands = d["candidates"]
248                 if not _is_fusion_schema(cands):
249                     skip_logger.info(skip_message, os.path.basename(path))
250                     continue
251                 if audio_policy == "require" and "audio" not in cands[0]:
252                     raise SystemExit(no_audio_error.format(path=path))
253                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
254                 features: Dict[str, List[float]] = {}
255                 for fn in shuttle_columns:
256                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
257                 for fn in person_columns:
258                     features[fn] = [float(c["person"][fn]) for c in cands]
259                 if audio_policy == "tolerate-missing":
260                     for fn in audio_columns:
261                         features[fn] = [
262                             float((c.get("audio") or {}).get(fn, 0.0)) for c in cands
263                         ]
264                 elif audio_policy == "require":
265                     for fn in audio_columns:
266                         features[fn] = [float(c["audio"][fn]) for c in cands]
267                 gts = [(float(a), float(b)) for a, b in d["gts"]]
268                 videos.append(

```



```

269         VideoCandidates(
270             name=d["name"], intervals=intervals, features=features, gts=gts
271         )
272     )
273     return videos
274
275
276     def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
277         """Persisted feature column, defaulting missing/short columns to 0.0 (matches
278         `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
279         col = vc.features.get(name)

```

34. assistant (2026-07-04T00:45:25.942Z)

Perfect! Now let me verify the schema difference more carefully. Let me look at the `\_is\_fusion\_schema` implementation in experiment.py more carefully and compare with `\_is\_full\_fusion\_feature\_file`:

35. user (2026-07-04T00:45:27.099Z)

```

def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
    """True iff every candidate row carries the per-cue feature dicts the golden-feature loaders
    read (`shuttle` + `person`; `audio` stays optional ? it is added later by
    `fusion_audio.augment` and older JSONs predate it).

    The velocity-windowing / distill corpus-growth path persists a LEANER
    ``*_golden_features.json`` row (`start`/`end` + velocity stats, no cue dicts) for clips
    that never went through fusion feature extraction. The loaders that build cue-rich
    ``VideoCandidates`` from these files ? `ablation.load_videos`,
    `fusion_compare.load_videos`
    and `fusion_audio.load_videos_with_audio` ? SKIP those rather than crash on
    `c["shuttle"]`,
    so a MIXED corpus (some fusion files, some windowing-only) loads the fusion subset cleanly.
    An empty candidate list is not a fusion file either.

    Single source of truth for the three loaders (consolidated 2026-06-18 after PRs #209/#210
    each landed an independent copy)."""
    return bool(candidates) and all(
        "shuttle" in c and "person" in c for c in candidates
    )

```

```

def load_golden_videos(

```

36. assistant (2026-07-04T00:45:30.311Z)

Now let me check the difference in behavior between the two functions. Let me also check if `load\_source\_video\_paths` is already present anywhere:

37. user (2026-07-04T00:45:31.165Z)

```

backend/eval/calibration.py:    `videos`: list of {"name": str, "predict_fn": PredictFn, "gts":
[Interval]}.
backend/eval/per_user_eval.py:    `heldout`. Held-out videos are video-aligned with the
returned list (caller pairs B?A)."""
tests/test_ablation.py: def _gate_videos(n: int = 6, *, separating: bool = True) ->
list[VideoCandidates]:
tests/test_ablation.py:     videos: list[VideoCandidates] = []
tests/test_ablation.py: def _learned_videos(n: int = 6) -> list[VideoCandidates]:
tests/test_ablation.py:     videos: list[VideoCandidates] = []
tests/test_api_endpoints.py: def test_api_videos_lists_all_newest_first(env):

```

38. assistant (2026-07-04T00:45:33.104Z)

Now let me verify if the `_filter_specs` function already exists anywhere else:

39. user (2026-07-04T00:45:34.344Z)

```
backend/eval/golden_manifest.py:          f"OK: {args.manifest} has {len(resolved)} video(s); "
```

40. assistant (2026-07-04T00:45:36.551Z)

Now let me analyze whether ``is_full_fusion_feature_file`` and ``_is_fusion_schema`` are duplicates. They differ in their implementation:

41. user (2026-07-04T00:45:39.156Z)

```
is_full_fusion_feature_file (fusion_golden.py):
- Takes a FILE PATH
- Opens and reads the JSON file
- Checks if it has shuttle+person cue dicts
- Returns True for empty candidates, False for non-list candidates
- Handles exceptions gracefully (returns False)

_is_fusion_schema (experiment.py):
- Takes a CANDIDATES LIST (already loaded)
- Checks if it has shuttle+person cue dicts
- Returns False for empty candidates
- Does NOT handle file I/O or exceptions
- Is already exported as part of the corpus loader pattern
- Comment says "Single source of truth for the three loaders"
- Used by: ablation.load_videos, fusion_compare.load_videos,
fusion_audio.load_videos_with_audio
```

42. assistant (2026-07-04T00:45:41.514Z)

Now let me create a comprehensive analysis document by checking for additional context. Let me verify if ``_is_fusion_schema`` is exposed as public or if we need a file-based version:

43. user (2026-07-04T00:45:42.761Z)

```
backend/eval/experiment.py- intervals: List[Interval]
backend/eval/experiment.py- features: Dict[str, List[float]] = field(default_factory=dict)
backend/eval/experiment.py- gts: Optional[List[Interval]] = None
backend/eval/experiment.py-
backend/eval/experiment.py-
backend/eval/experiment.py:def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
--
tests/test_ablation.py- AblationReport,
tests/test_ablation.py- Signal,
tests/test_ablation.py- ablate_one,
tests/test_ablation.py- run_ablation,
tests/test_ablation.py-)
tests/test_ablation.py:from backend.eval.experiment import Variant, VideoCandidates,
_is_fusion_schema
```

44. assistant (2026-07-04T00:45:44.850Z)

Good! ``_is_fusion_schema`` is already imported in tests. Now let me verify the exact usage in the test to understand the scope:

45. user (2026-07-04T00:45:45.862Z)

```
from backend.eval.experiment import Variant, VideoCandidates, _is_fusion_schema
```

----- synthetic fixtures

```
def test_ablation_module_help_runs_without_import_cycle():
    --
    if _is_fusion_schema(
        cands
    ): # canonical predicate (SSOT in backend.eval.experiment, #215)
        n += 1
    return n >= 6
```

46. assistant (2026-07-04T00:45:47.919Z)

Perfect. Now let me check if there are any other path-resolution functions I should look for:

47. user (2026-07-04T00:45:49.138Z)

```
backend/eval/batch.py:Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted
by
backend/eval/batch.py:def resolve_video_path(local_path: Optional[str], videos_root: str) ->
Optional[str]:
backend/eval/fusion_audio.py:    _derive_video_path,
backend/eval/fusion_audio.py:    attach_source_video_paths,
backend/eval/fusion_audio.py:    updated = attach_source_video_paths(specs, source_manifest)
backend/eval/fusion_audio.py:    video = _derive_video_path(spec)
backend/eval/fusion_audio.py:    "collector golden_source_videos.json; overrides video
paths for source/proxy "
backend/eval/fusion_golden.py:def load_source_video_paths(source_manifest_path: Optional[str])
-> Dict[str, str]:
backend/eval/fusion_golden.py:def attach_source_video_paths(
backend/eval/fusion_golden.py:    """Attach explicit `video_path` fields to manifest specs
from the source manifest.
backend/eval/fusion_golden.py:    source_paths = load_source_video_paths(source_manifest_path)
backend/eval/fusion_golden.py:    if spec.get("video_path") == source_path:
backend/eval/fusion_golden.py:    spec["video_path"] = source_path
backend/eval/fusion_golden.py:def _derive_video_path(spec: Dict[str, Any]) -> str:
backend/eval/fusion_golden.py:    """Original MP4 for a manifest entry ? explicit `video_path`,
else derived from the
backend/eval/fusion_golden.py:    if spec.get("video_path"):
backend/eval/fusion_golden.py:    return str(spec["video_path"])
backend/eval/fusion_golden.py:    f"{spec.get('name')}: no video_path and labels {labels!r}
isn't *.rallies.csv"
backend/eval/fusion_golden.py:    video_path = _derive_video_path(spec)
backend/eval/fusion_golden.py:    if not os.path.isfile(video_path):
```

48. assistant (2026-07-04T00:45:51.365Z)

Interesting! There's a `resolve\_video\_path` in batch.py. Let me check that:

49. user (2026-07-04T00:45:52.143Z)

Segmenters that only ever populate the `final` play_segments table (no raw candidate stage), so scoring `candidates` for them would be meaninglessly empty.

```
_FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}
```

```
def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
    """Resolve a manifest `local_path` to an absolute path.

    The collector stores paths like `./data/videos/shuttleaset_1.mp4` relative
    to its own repo root and with Windows separators. Normalise separators, strip
    a leading `./`, and resolve relative paths against `videos_root` (the
    collector root). Returns None for a missing/empty path.
    """
    if not local_path:
        return None
```

```

p = local_path.replace("\\", "/")
if p.startswith("./"):
    p = p[2:]
# Cross-platform: treat Windows drive-letter absolute paths (C:/...) as absolute
# even when running on Linux (os.path.isabs("C:/...") == False on Unix).
# This keeps collector manifest paths working in tests and mixed environments.

```

50. assistant (2026-07-04T00:45:55.261Z)

Let me see the full function:

51. user (2026-07-04T00:45:56.057Z)

```

def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
    """Resolve a manifest `local_path` to an absolute path.

    The collector stores paths like ``./data/videos\\shuttleset_1.mp4`` relative
    to its own repo root and with Windows separators. Normalise separators, strip
    a leading ``./``, and resolve relative paths against `videos_root` (the
    collector root). Returns None for a missing/empty path.
    """
    if not local_path:
        return None
    p = local_path.replace("\\", "/")
    if p.startswith("./"):
        p = p[2:]
    # Cross-platform: treat Windows drive-letter absolute paths (C:/...) as absolute
    # even when running on Linux (os.path.isabs("C:/...") == False on Unix).
    # This keeps collector manifest paths working in tests and mixed environments.
    if (len(p) > 1 and p[1] == ":" and p[0].isalpha()) or os.path.isabs(p):
        return os.path.normpath(p)
    return os.path.normpath(os.path.join(videos_root, p))

def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
    """Pick which prediction stages to score.

    ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
    but only ``final`` for segmenters that don't emit candidates. An explicit
    """

```

52. assistant (2026-07-04T00:45:58.265Z)

Now let me verify the full picture. Let me look at what PR #452 (golden_manifest.py) actually did regarding video path handling:

53. user (2026-07-04T00:45:59.444Z)

```

a68f02e eval: add portable golden manifest check (#452)
7f6f7c5 eval: add portable golden manifest check
e585965 Fix ffmpeg concat clip paths (#449)
959c630 fix(#397): write per-video report to the resolved local path, not the raw URI (#400)
a96006f fix: D12 sys.path hack + B3 PTS gap config + 05 lock note (#364)
b25fdb8 docs: explain lazy cv2 import in --debug-clip path (Q8) (#361)
598c865 refactor: deduplicate _default_db_path into shared helper (Q3) (#356)
6d4514c fix: WASB cache manifest-compat tests on Windows (#353)
81b0cfc fix(wasb): don't abspath the cache key in manifest_compatible (#348) (#350)
239c8eb fix(scripts): resolve gcloud via PATH in verify_compute_path so --verify-gcp works on
Windows (#344)

```

54. assistant (2026-07-04T00:46:13.900Z)

Now I have enough information. Let me create the final analysis by compiling all findings into JSON format:

```

...json
[
  {
    "file": "backend/eval/experiment.py",
    "line": 187,
    "summary": "experiment._is_fusion_schema() already checks shuttle+person cue dicts; is_full_fusion_feature_file() in fusion_golden.py (line 42) duplicates this schema check but adds file I/O. The test imports _is_fusion_schema as SSOT (test_ablation.py:230 references '#215'). Consider wrapping _is_fusion_schema in a file-based helper or exposing it publicly.",
    "failure_scenario": "Three schema-validation functions now exist: _is_fusion_schema (on candidates list), is_full_fusion_feature_file (file path), and _is_fusion_schema hidden in experiment. When the schema definition changes (e.g., audio becomes required), maintainers must update both locations, risking drift."
  },
  {
    "file": "backend/eval/golden_manifest.py",
    "line": 99,
    "summary": "golden_manifest.py (lines 99-197) already has path-resolution logic: resolve_manifest_paths(), resolve_existing_paths(). The new attach_source_video_paths() (fusion_golden.py:97) adds source-video rebasing from a collector manifest. This competes with the path-resolution pattern established in A2 (PR #452). Consider whether source-video overrides should be integrated into resolve_manifest_specs() instead of a separate attach step.",
    "failure_scenario": "Two competing path-resolution mechanisms: golden_manifest rebases trajectory/labels fields on load; attach_source_video_paths mutates specs in-place after loading. Callers must remember to call both in the right order. Future video path sources (e.g., WASB catalog) will add a third mechanism."
  },
  {
    "file": "backend/eval/ablation.py",
    "line": 453,
    "summary": "ablation.load_videos() (line 453) already skips leaner windowing-only JSONs via _is_fusion_schema. The new is_full_fusion_feature_file() in fusion_golden.py does the same check but file-first. Consider whether ablation's skip logic should reuse experiment._is_fusion_schema or whether fusion_golden should use the same pattern.",
    "failure_scenario": "Two implementations of the schema check (candidate-list vs file-based) will diverge when edge cases surface (e.g., empty files, corrupted JSON). A mixed corpus loaded by ablation will skip files differently than fusion_golden would."
  },
  {
    "file": "backend/eval/fusion_golden.py",
    "line": 120,
    "summary": "_filter_specs() (line 120) is a simple --only name filter copied to fusion_audio.py (line 29). No sibling eval CLI has this pattern yet, but it is trivial and specific to manifest-driven CLIs. Consider whether this warrants a shared utils function or is acceptable as small duplication.",
    "failure_scenario": "Each new eval CLI will duplicate _filter_specs if they add --only filtering. Error messages and filter logic will diverge (e.g., case sensitivity, substring vs exact match, handling of duplicates)."
  },
  {
    "file": "backend/eval/batch.py",
    "line": 70,
    "summary": "batch.resolve_video_path() (line 70) normalizes collector paths (Windows backslashes, relative ./ prefixes, drive-letter handling). The new load_source_video_paths() (fusion_golden.py:68) reads local_path directly from golden_source_videos.json without the cross-platform normalization. Unclear if fusion paths need normalization or if batch.py's logic is a one-time shim.",
    "failure_scenario": "If golden_source_videos.json paths use Windows separators (as batch.py expects from the collector), fusion_golden paths may be non-canonical on Linux, causing isfile() checks to fail silently or match incorrectly."
  }
]
...

```

Let me verify the actual file paths and make sure the line numbers are correct in the PR:

55. user (2026-07-04T00:46:14.983Z)

```
99: def resolve_manifest_paths(
```

56. assistant (2026-07-04T00:46:17.029Z)

Perfect. Now let me verify the batch.py line number:

57. user (2026-07-04T00:46:17.881Z)

```
21: def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
```

58. assistant (2026-07-04T00:46:25.018Z)

Let me finalize the analysis JSON with the correct line numbers:

```
[
  {
    "file": "backend/eval/experiment.py",
    "line": 187,
    "summary": "_is_fusion_schema() already checks shuttle+person cue dicts;
is_full_fusion_feature_file() in fusion_golden.py duplicates this schema check but adds file
I/O. Test imports _is_fusion_schema as SSOT (test_ablation.py references '#215'). Consider
wrapping _is_fusion_schema in a file-based helper or exposing it publicly.",
    "failure_scenario": "Three schema-validation functions now exist: _is_fusion_schema (on
candidates list), is_full_fusion_feature_file (file path), and _is_fusion_schema hidden in
experiment. When the schema definition changes (e.g., audio becomes required), maintainers must
update both locations, risking drift."
  },
  {
    "file": "backend/eval/golden_manifest.py",
    "line": 99,
    "summary": "resolve_manifest_paths() already has path-resolution logic for
trajectory/labels. The new attach_source_video_paths() in fusion_golden.py adds source-video
rebasings from a collector manifest. This competes with the path-resolution pattern established
in PR #452. Consider whether source-video overrides should be integrated into
resolve_manifest_specs() instead of a separate attach step.",
    "failure_scenario": "Two competing path-resolution mechanisms: golden_manifest rebases
trajectory/labels fields on load; attach_source_video_paths mutates specs in-place after
loading. Callers must remember to call both in the right order. Future video path sources will
add a third mechanism."
  },
  {
    "file": "backend/eval/batch.py",
    "line": 21,
    "summary": "resolve_video_path() normalizes collector paths (Windows backslashes, relative
./ prefixes, drive-letter handling). The new load_source_video_paths() in fusion_golden.py reads
local_path directly from golden_source_videos.json without cross-platform normalization. Unclear
if fusion paths need normalization.",
    "failure_scenario": "If golden_source_videos.json paths use Windows separators (as batch.py
expects from the collector), fusion_golden paths may be non-canonical on Linux, causing isfile()
checks to fail silently or match incorrectly."
  }
]

```json
[
 {
 "file": "backend/eval/experiment.py",
 "line": 187,
 "summary": "_is_fusion_schema() already checks shuttle+person cue dicts;
is_full_fusion_feature_file() in fusion_golden.py duplicates this schema check but adds file
I/O. Test imports _is_fusion_schema as SSOT (test_ablation.py references '#215'). Consider
wrapping _is_fusion_schema in a file-based helper.",
```

"failure\_scenario": "Three schema-validation functions now exist. When the schema definition changes, maintainers must update both locations, risking drift."

},

{

"file": "backend/eval/golden\_manifest.py",

"line": 99,

"summary": "resolve\_manifest\_paths() already has path-resolution logic. The new attach\_source\_video\_paths() in fusion\_golden.py adds source-video rebasing from a collector manifest, competing with the PR #452 path-resolution pattern. Consider integrating into resolve\_manifest\_specs().",

"failure\_scenario": "Two competing path-resolution mechanisms mutate specs differently. Callers must remember to call both in the right order. Future video path sources will add a third mechanism."

},

{

"file": "backend/eval/batch.py",

"line": 21,

"summary": "resolve\_video\_path() normalizes collector paths (Windows backslashes, relative ./ prefixes, drive-letter handling). The new load\_source\_video\_paths() in fusion\_golden.py reads local\_path directly without cross-platform normalization.",

"failure\_scenario": "If golden\_source\_videos.json paths use Windows separators, fusion\_golden paths may be non-canonical on Linux, causing isfile() checks to fail silently."

}

]

...